

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

Lab 7. Mạng Nơ-ron Tích Chập (CNN)

Cảm nhận trước – Lý thuyết sau

Hà Minh Tuấn

Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 13 tháng 5 năm 2026

Giới thiệu bài thực hành

Giúp sinh viên làm quen CNN, trong đó có **nhiều khái niệm trừu tượng**: convolution, kernel, feature map, receptive field, pooling, backpropagation qua tích chập

Bài thực hành này gồm 2 giai đoạn

- **Giai đoạn 1 – WOW phase**: huấn luyện CNN trên MNIST/CIFAR, thấy accuracy tăng từng epoch, xem mô hình nhận đúng/sai từng ảnh, thử đổi số lớp, thử augmentation. **Chưa cần giải thích sâu** – chỉ cần thấy “AI thực sự hoạt động được”.
- **Giai đoạn 2 – Intuition phase**: dùng *hình ảnh, không phải công thức*. Thấy kernel trượt qua ảnh để *phát hiện cạnh*; thấy pooling *nép* thông tin; thấy mỗi lớp sâu hơn nhìn thấy thứ phức tạp hơn (cạnh → kết cấu → bộ phận → vật thể).

Nội dung lý thuyết sẽ được học sau

Mục tiêu cụ thể

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- **Tự huấn luyện được** một CNN trên MNIST/CIFAR-10 chỉ với 30–50 dòng code, thấy accuracy tăng theo epoch.
- **Cảm nhận bằng mắt** được kernel làm gì với ảnh: phát hiện cạnh ngang/dọc, làm mờ, làm nét, embossing.

- **Quan sát feature map** ở các lớp khác nhau: lớp đầu thấy *cạnh* và *texture*; lớp sâu thấy *bộ phận* và *vật thể*.
- Dùng được **CNN Explainer** (công cụ trực quan của Poloclub) để “nhìn xuyên qua” một CNN đang hoạt động.
- Tự đặt ra được những câu hỏi như: “*Vì sao CNN học được các filter này?*”, “*Padding và stride ảnh hưởng kích thước thế nào?*” – sẽ được trả lời ở buổi lý thuyết tiếp theo.

Công cụ & Môi trường

- **Ngôn ngữ:** Python 3.9+ với PyTorch ≥ 2.0 , torchvision ≥ 0.15 .
- **Nền tảng:** Google Colab (GPU T4 miễn phí) hoặc Kaggle Notebook. **Bật GPU trước khi chạy:** Runtime $\rightarrow$ Change runtime type $\rightarrow$ T4 GPU.
- **Thư viện cần cài (Colab có sẵn hết):**

```
!pip install torch torchvision matplotlib numpy pillow -q
```

1. GIAI ĐOẠN 1 – WOW phase: Cho CNN chạy thật

Mục tiêu Giai đoạn 1

Trong ~30 phút đầu của buổi thực hành, sinh viên sẽ:

1. Chạy được một CNN trên MNIST, thấy **accuracy nhảy từ 10% lên 98%** qua vài epoch.
2. Visualize **từng dự đoán**: xem ảnh nào đúng, ảnh nào sai, mô hình “tự tin” bao nhiêu.
3. Thử **thay đổi số lớp** của CNN – thấy mô hình sâu hơn / nông hơn ảnh hưởng thế nào.
4. Bật/tắt **data augmentation** – thấy accuracy thay đổi.

Sinh viên **không cần hiểu Conv2d hoạt động ra sao** ở giai đoạn này. Chỉ cần copy-chạy-quan sát.

1.1. Bước 1 – Chuẩn bị môi trường (1 phút)

Cell 1: Import

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

# Tái lập kết quả
torch.manual_seed(42)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Đang dùng: {device}")
```

Câu hỏi 1.1 – Khởi động

- (a) Output có in ra cuda không? Nếu in cpu, vào menu *Runtime* của Colab và chuyển sang GPU T4. Tại sao GPU lại nhanh hơn CPU rất nhiều khi train

CNN? (Chưa cần trả lời chi tiết – một câu cảm nhận là đủ.)

(b) `torch.manual_seed(42)` dùng để làm gì?

1.2. Bước 2 – Tải dữ liệu MNIST và xem thử

Cell 2: Tải MNIST và xem một ít ảnh

```
transform = transforms.Compose([
    transforms.ToTensor(),                # PIL -> tensor [0,
    ↪ 1]
    transforms.Normalize((0.1307,), (0.3081,)) # Trung bình/std
    ↪ MNIST
])

train_set = datasets.MNIST(root='./data', train=True,
                           download=True, transform=transform)
test_set = datasets.MNIST(root='./data', train=False,
                           download=True, transform=transform)

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
test_loader = DataLoader(test_set, batch_size=64, shuffle=False)

# Lấy 1 batch và hiển thị
images, labels = next(iter(train_loader))
print(f"Batch images shape: {images.shape}")
print(f"Batch labels shape: {labels.shape}")

fig, axs = plt.subplots(2, 8, figsize=(12, 3.2))
for i, ax in enumerate(axs.flat):
    ax.imshow(images[i, 0], cmap='gray')
    ax.set_title(f"label = {labels[i].item()}")
    ax.axis('off')
plt.tight_layout(); plt.show()
```

Câu hỏi 2.1 – Quan sát data

- (a) Shape của `images` là `torch.Size([64, 1, 28, 28])`. Đoán xem 4 con số này nghĩa là gì?
- (b) Tại sao là **1 kênh màu** mà không phải 3?
- (c) Thử in `train_set.classes` và `len(train_set)`, `len(test_set)`. Có bao

nhiều ảnh để train, bao nhiêu để test?

1.3. Bước 3 – Tạo một CNN nhỏ (chưa cần hiểu)

Cell 3: Định nghĩa CNN – chỉ copy thôi

```
class TinyCNN(nn.Module):
    """CNN tối giản 2 lớp tích chập + 1 lớp fully-connected."""
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)    # 1 -> 16
        ↪ kênh
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)  # 16 -> 32
        ↪ kênh
        self.pool = nn.MaxPool2d(2, 2)                          # giảm 1/2
        self.fc = nn.Linear(32 * 7 * 7, 10)                     # 10 lớp
        ↪ 0..9

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))    # 28x28 -> 14x14
        x = self.pool(F.relu(self.conv2(x)))    # 14x14 -> 7x7
        x = x.view(x.size(0), -1)              # flatten thành vector
        x = self.fc(x)
        return x

model = TinyCNN().to(device)
print(model)
print(f"Tổng tham số: {sum(p.numel() for p in model.parameters()):,}")
```

Đừng cố hiểu code này lúc này!

Sinh viên **chưa cần hiểu** Conv2d, MaxPool2d, hay tại sao là $32 * 7 * 7$. Hãy coi đây là một **hộp đen có khả năng học**: ta cho ảnh vào, nó trả ra dự đoán. Phần “tại sao” sẽ được giải thích đầy đủ ở các buổi sau. Bây giờ chỉ cần biết: nó **hoạt động**.

1.4. Bước 4 – TRAIN!

Cell 4: Vòng lặp huấn luyện

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_acc_history, test_acc_history = [], []

for epoch in range(3):                                     # Chỉ 3 epoch là đủ để
    ↪ WOW
    # ----- TRAIN -----
    model.train()
    correct, total = 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        correct += (outputs.argmax(1) == labels).sum().item()
        total   += labels.size(0)
    train_acc = 100 * correct / total

    # ----- TEST -----
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            correct += (outputs.argmax(1) == labels).sum().item()
            total   += labels.size(0)
    test_acc = 100 * correct / total

    train_acc_history.append(train_acc)
    test_acc_history.append(test_acc)
    print(f"Epoch {epoch+1}: train_acc = {train_acc:.2f}% | test_acc =
    ↪ {test_acc:.2f}%")
```

Khoảnh khắc WOW

Sau 3 epoch (~2–3 phút trên GPU T4), sinh viên sẽ thấy:

Epoch 1: train_acc = 95.20% | test_acc = 97.85%

Epoch 2: train_acc = 98.40% | test_acc = 98.50%

Epoch 3: train_acc = 98.95% | test_acc = 98.78%

Mô hình đã đoán đúng ~98–99% trong 10.000 ảnh chữ số viết tay mà chưa từng thấy bao giờ.

Câu hỏi 4.1 – Phản ứng đầu tiên

- (a) Accuracy ban đầu (trước train) là khoảng bao nhiêu %? Thử thêm dòng đánh giá trước vòng lặp `for epoch` và in ra. (Gợi ý: nên gần 10% – vì có 10 lớp nên đoán đại = $1/10$.)
- (b) Hãy nhắc lại công thức tính Accuracy cho bài toán phân loại đa lớp.
- (c) Thử train thêm 2 epoch nữa (tổng 5). Accuracy có còn tăng không? Tăng nhanh hay chậm dần?

1.5. Bước 5 – Vẽ biểu đồ accuracy theo epoch

Cell 5: Visualize learning curve

```
plt.figure(figsize=(7, 4))
plt.plot(train_acc_history, 'o-', label='Train accuracy')
plt.plot(test_acc_history, 's-', label='Test accuracy')
plt.xlabel('Epoch'); plt.ylabel('Accuracy (%)')
plt.title('CNN học MNIST -- accuracy tăng theo epoch')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Câu hỏi 5.1 – Đường cong học

- (a) Train accuracy và test accuracy cái nào cao hơn? Đây là điều bình thường hay bất thường?
- (b) Ở epoch nào accuracy **tăng nhanh nhất**? Ở các epoch sau, đường cong có *phẳng dần* không?

1.6. Bước 6 – Xem từng dự đoán cụ thể

Cell 6: Visualize prediction

```
model.eval()
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)
with torch.no_grad():
    outputs = model(images)
    probs = F.softmax(outputs, dim=1)
    preds = outputs.argmax(1)

# 16 ảnh đầu của batch
fig, axs = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axs.flat):
    img = images[i, 0].cpu().numpy()
    pred, true = preds[i].item(), labels[i].item()
    conf = probs[i, pred].item() * 100
    color = 'green' if pred == true else 'red'
    ax.imshow(img, cmap='gray')
    ax.set_title(f"pred={pred} ({conf:.0f}%) \n true={true}",
                color=color, fontsize=9)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

Cell 6 (tiếp): Tìm những ảnh mô hình đoán SAI

```
# Quét toàn bộ test set, gom các ảnh đoán sai
wrong_imgs, wrong_pred, wrong_true = [], [], []
model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(1)
        mask = preds != labels
        for img, p, t in zip(images[mask], preds[mask], labels[mask]):
            wrong_imgs.append(img.cpu()); wrong_pred.append(p.item())
            wrong_true.append(t.item())
        if len(wrong_imgs) >= 16: break

# Hiển thị 16 ảnh sai đầu tiên
fig, axs = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axs.flat):
```



```

if i < len(wrong_imgs):
    ax.imshow(wrong_imgs[i][0], cmap='gray')
    ax.set_title(f"pred={wrong_pred[i]}\ntrue={wrong_true[i]}",
                 color='red', fontsize=9)
    ax.axis('off')
plt.suptitle("Những ảnh CNN đoán SAI", color='red')
plt.tight_layout()
plt.show()

```

Câu hỏi 6.1 – Mô hình thấy gì khi sai?

- (a) Quan sát 16 ảnh đoán sai. Em có thể nhận ra số đó là gì không, kể cả khi là người? Có những ảnh viết *rất xấu/rất kỳ*?
- (b) Cặp số nào hay bị nhầm lẫn? (*Thường là 4-9, 3-5, 7-1, hoặc 1-7.*) Hai số đó có nét chữ giống nhau không?
- (c) **Cảm giác:** mô hình “sai” theo cách *có lý* hay *vô lý*? Đây là một dấu hiệu cho thấy CNN không học vệt mà thực sự học *đặc trưng*.

1.7. Bước 7 – Thử nghiệm: đổi số lớp của CNN

Cell 7: 3 mạng – nông, vừa, sâu

```

# (a) MẠNG NÔNG: 1 lớp Conv duy nhất
class ShallowCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(16 * 14 * 14, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv(x)))
        return self.fc(x.view(x.size(0), -1))

# (b) MẠNG VỪA: 2 lớp Conv (đã train ở Bước 3-4)
# class TinyCNN: ...

# (c) MẠNG SÂU: 3 lớp Conv
class DeepCNN(nn.Module):
    def __init__(self):
        super().__init__()

```

```

        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(64 * 3 * 3, 10)           # 28 -> 14 -> 7 -> 3
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        return self.fc(x.view(x.size(0), -1))

# Hàm train ngắn gọn để so sánh
def quick_train(model_class, epochs=3):
    m = model_class().to(device)
    opt = optim.Adam(m.parameters(), lr=0.001)
    crit = nn.CrossEntropyLoss()
    for ep in range(epochs):
        m.train()
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            opt.zero_grad(); crit(m(x), y).backward(); opt.step()

# Đánh giá
m.eval(); correct, total = 0, 0
with torch.no_grad():
    for x, y in test_loader:
        x, y = x.to(device), y.to(device)
        correct += (m(x).argmax(1) == y).sum().item()
        total += y.size(0)
n_params = sum(p.numel() for p in m.parameters())
print(f"{model_class.__name__:11s}: ``
      f"params = {n_params:>8,} | test_acc = {100*correct/total:.2f}%")

print("Huấn luyện 3 kiến trúc khác nhau, 3 epoch mỗi mạng...\n")
quick_train(ShallowCNN)
quick_train(TinyCNN)
quick_train(DeepCNN)

```

Câu hỏi 7.1 – Sâu hơn = tốt hơn?

- (a) Mạng nào **nhiều tham số nhất**? Là mạng nông, vừa, hay sâu?
- (b) Mạng nào **accuracy cao nhất**? Có phải **càng sâu càng tốt** không? Hay có sự bão hòa?
- (c) (Câu hỏi mở – không có đáp án “đúng” duy nhất) Tại sao mạng sâu hơn lại học tốt hơn? Em **đoán** thử lý do? Câu trả lời chính thức sẽ ở buổi lý thuyết.

1.8. Bước 8 – Thử nghiệm: bật/tắt augmentation

Cell 8: Data augmentation đơn giản

```
# Augmentation: xoay nhẹ + dịch chuyển ngẫu nhiên
transform_aug = transforms.Compose([
    transforms.RandomRotation(degrees=10),
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Xem một ảnh được augment 8 lần khác nhau
sample_idx = 0
raw_set = datasets.MNIST(root='./data', train=True, download=True,
                        transform=transform_aug)

fig, axs = plt.subplots(1, 8, figsize=(14, 2))
for i, ax in enumerate(axs):
    img, lbl = raw_set[sample_idx]    # gọi lại nhiều lần -> ra ảnh khác nhau
    ax.imshow(img[0], cmap='gray')
    ax.set_title(f"aug #{i+1}", fontsize=8)
    ax.axis('off')
plt.suptitle(f"1 ảnh số {lbl} -- 8 lần augment khác nhau")
plt.tight_layout(); plt.show()
```

Cell 8 (tiếp): Train với augmentation

```
train_set_aug = datasets.MNIST(root='./data', train=True, download=True,
                              transform=transform_aug)
train_loader_aug = DataLoader(train_set_aug, batch_size=64, shuffle=True)

# Train lại TinyCNN với data đã augment
m_aug = TinyCNN().to(device)
```

```

opt = optim.Adam(m_aug.parameters(), lr=0.001)
crit = nn.CrossEntropyLoss()

for ep in range(3):
    m_aug.train()
    for x, y in train_loader_aug:
        x, y = x.to(device), y.to(device)
        opt.zero_grad(); crit(m_aug(x), y).backward(); opt.step()

m_aug.eval(); correct, total = 0, 0
with torch.no_grad():
    for x, y in test_loader:
        x, y = x.to(device), y.to(device)
        correct += (m_aug(x).argmax(1) == y).sum().item()
        total += y.size(0)

print(f"TinyCNN + augmentation: test_acc = {100*correct/total:.2f}%")
print(f"TinyCNN không augment (so sánh): {test_acc_history[-1]:.2f}%")

```

Câu hỏi 8.1 – Augmentation giúp gì?

- Khi xem 8 phiên bản augment của *cùng một ảnh*, các ảnh **khác nhau hay giống nhau**? Người vẫn dễ dàng nhận ra số bao nhiêu chữ?
- Trên MNIST, augmentation có giúp accuracy tăng nhiều không? Trên dataset *nhỏ và sạch* như MNIST, hiệu ứng có thể không lớn. Augmentation thường **hữu ích hơn** trên dataset *phức tạp* (như CIFAR-10, ImageNet). Hãy nhớ điều này cho Phần 2.

Tổng kết Giai đoạn 1 – WOW phase

Sau khoảng 25–30 phút, sinh viên đã:

- Tự tay train được CNN đạt $> 98\%$ trên MNIST.
- Thấy được *learning curve* đi lên theo epoch.
- Quan sát được *ảnh đoán đúng/sai* của mô hình.
- Thử nghiệm với *số lớp khác nhau* và *augmentation*.

Đến lúc này, mọi sinh viên đều có cảm giác: “OK, CNN làm được những gì tôi vừa thấy. *Nhưng làm thế nào?*”

Câu hỏi đó là cánh cửa vào **Giai đoạn 2**.

2. GIAI ĐOẠN 2 – Intuition phase: Tổng thể CNN

Mục tiêu Giai đoạn 2

Trong ~ 45 phút tiếp theo, sinh viên sẽ **nhìn thấy bằng mắt** – không phải bằng công thức – **ba khái niệm cốt lõi của CNN**:

1. **Kernel quét ảnh**: thấy một filter 3×3 *trượt qua* ảnh và tạo ra ảnh mới phát hiện cạnh, làm mờ, làm nét, v.v.
2. **Pooling**: thấy phép pooling *nép* ảnh nhưng vẫn *giữ thông tin chính*.
3. **Feature hierarchy**: thấy lớp đầu của VGG-16 *phát hiện cạnh*, lớp giữa *phát hiện kết cấu*, lớp sâu *phát hiện bộ phận và vật thể*.

Vẫn không cần công thức ở giai đoạn này. Toán sẽ ở các buổi sau.

2.1. Trực giác 1: Kernel quét ảnh – Edge Detection thử công

Ý tưởng cốt lõi

Một **kernel** là một ma trận nhỏ (thường 3×3). Khi ta *trượt* kernel này qua ảnh và *nhân điểm-điểm rồi cộng*, ta được một **ảnh mới** – gọi là *feature map*. Kernel khác nhau cho ra ảnh mới khác nhau:

- Kernel “cạnh ngang” $\rightarrow$ ảnh mới chỉ **sáng ở vị trí có cạnh ngang**.
- Kernel “làm mờ” $\rightarrow$ ảnh mới *mờ đi*.
- Kernel “làm nét” $\rightarrow$ ảnh mới *rõ hơn*.

Trong CNN: các kernel không được người gán mà **tự học từ dữ liệu**. Đó là phép màu.

Cell 9: Áp dụng các kernel cổ điển lên một ảnh

```
import torch
import torch.nn.functional as F
from PIL import Image
import urllib.request, io
import matplotlib.pyplot as plt
import numpy as np

# Tải ảnh: dùng ảnh sample của torchvision (con mèo) hoặc ảnh tự upload
```

```

url = "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"
#img = Image.open(io.BytesIO(urllib.request.urlopen(url).read()))
req = urllib.request.Request(
    url,
    headers={"User-Agent": "Mozilla/5.0"}
)

img = Image.open(io.BytesIO(
    urllib.request.urlopen(req).read()
))

img = img.convert('L').resize((256, 256)) # grayscale 256x256
img_t = torch.tensor(np.array(img), dtype=torch.float32)
img_t = img_t.unsqueeze(0).unsqueeze(0) / 255. # [1,1,256,256] in
↪ [0,1]

# Định nghĩa các kernel cổ điển
kernels = {
    "Identity (không đổi)": torch.tensor([[ 0, 0, 0],
                                           [ 0, 1, 0],
                                           [ 0, 0, 0]]),
    "Cạnh ngang (Sobel-y)": torch.tensor([[ -1, -2, -1],
                                           [ 0, 0, 0],
                                           [ 1, 2, 1]]),
    "Cạnh dọc (Sobel-x)": torch.tensor([[ -1, 0, 1],
                                         [ -2, 0, 2],
                                         [ -1, 0, 1]]),
    "Làm mờ (box blur)": torch.tensor([[ 1, 1, 1],
                                         [ 1, 1, 1],
                                         [ 1, 1, 1]]) / 9.0,
    "Làm nét (sharpen)": torch.tensor([[ 0, -1, 0],
                                         [ -1, 5, -1],
                                         [ 0, -1, 0]]),
    "Emboss": torch.tensor([[ -2, -1, 0],
                             [ -1, 1, 1],
                             [ 0, 1, 2]]),
}

fig, axs = plt.subplots(2, 4, figsize=(14, 7))
axs[0, 0].imshow(img, cmap='gray'); axs[0, 0].set_title("Ảnh gốc")

```

```

axs[0, 0].axis('off')

# Áp dụng từng kernel
for ax, (name, K) in zip(axs.flat[1:], kernels.items()):
    K = K.float().view(1, 1, 3, 3)
    out = F.conv2d(img_t, K, padding=1).squeeze().numpy()
    ax.imshow(out, cmap='gray'); ax.set_title(name)
    ax.axis('off')

# Tắt subplot dư
for i in range(len(kernels) + 1, 8):
    axs.flat[i].axis('off')

plt.tight_layout(); plt.show()

```

Khoảnh khắc thấy được

Sinh viên **nhìn thấy bằng mắt**: cùng một ảnh con mèo, nhưng:

- Sobel-x cho ra một bức **chỉ có cạnh dọc** (lông mèo dựng đứng).
- Sobel-y cho ra một bức **chỉ có cạnh ngang** (đường viền mí mắt, miệng).
- Box blur cho ra ảnh *nhòe đi* – giống như xem qua kính cận.
- Sharpen làm *rõ nét* hơn ảnh gốc.

Một con số 3×3 thay đổi hoàn toàn ý nghĩa của ảnh! Đó chính là *convolution*.

Câu hỏi 9.1 – Cảm nhận kernel

- Nhìn vào kernel “Sobel-y” = $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$. **Đoán xem** vì sao nó phát hiện được cạnh *ngang*? (*Hint: vùng trên kernel có giá trị âm, vùng dưới dương – nó tính độ chênh lệch sáng giữa trên và dưới.*)
- Tự thiết kế một kernel “phát hiện đường chéo /” (chéo từ dưới-trái lên trên-phải). (*Một gợi ý: $\begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$.*)
- Trong CNN, các kernel này **không phải do người tạo ra**. Mạng tự học chúng từ data. **Cảm giác đó như thế nào?** (Câu hỏi suy ngẫm.)

2.2. Trực giác 2: Pooling – Nén ảnh nhưng giữ ý nghĩa

Cell 10: Pooling – thấy ảnh thu nhỏ

```
# Áp dụng MaxPool và AvgPool, mỗi lần size giảm 1/2
sizes = [256, 128, 64, 32, 16]
fig, axs = plt.subplots(2, 5, figsize=(14, 6))

x = img_t.clone()
axs[0, 0].imshow(x[0, 0], cmap='gray')
axs[0, 0].set_title(f"Gốc -- {x.shape[-1]}x{x.shape[-1]}")
axs[1, 0].imshow(x[0, 0], cmap='gray')
axs[1, 0].set_title(f"Gốc -- {x.shape[-1]}x{x.shape[-1]}")

# MaxPool dần dần
xm = img_t.clone()
for i in range(4):
    xm = F.max_pool2d(xm, 2)
    axs[0, i+1].imshow(xm[0, 0], cmap='gray')
    axs[0, i+1].set_title(f"MaxPool x{i+1}\n{xm.shape[-1]}x{xm.shape[-1]}")

# AvgPool dần dần
xa = img_t.clone()
for i in range(4):
    xa = F.avg_pool2d(xa, 2)
    axs[1, i+1].imshow(xa[0, 0], cmap='gray')
    axs[1, i+1].set_title(f"AvgPool x{i+1}\n{xa.shape[-1]}x{xa.shape[-1]}")

for ax in axs.flat: ax.axis('off')
axs[0, 0].set_ylabel("MaxPool", fontsize=12)
axs[1, 0].set_ylabel("AvgPool", fontsize=12)
plt.suptitle("Pooling -- nén ảnh từng bước, vẫn nhận ra con mèo")
plt.tight_layout(); plt.show()
```

Câu hỏi 10.1 – Pooling

- (a) Sau 4 lần pool, ảnh từ 256×256 co lại còn $? \times ?$. Em vẫn nhận ra con mèo ở kích thước nhỏ nhất chứ?
- (b) So sánh hai dòng MaxPool và AvgPool: cái nào trông "nét" hơn, cái nào trông "mờ" hơn?
- (c) Vì sao CNN dùng pooling? (Đoán) Có ít nhất 2 lý do: (i) làm _____, (ii)

tăng _____. Đáp án: (i) giảm kích thước feature map → giảm tính toán; (ii) tăng *receptive field* (vùng nhìn) của các lớp sâu hơn – mỗi pixel ở lớp sâu “nhìn thấy” nhiều pixel ảnh gốc hơn.

2.3. Trực giác 3: Feature Hierarchy – Lớp càng sâu, “nhìn” càng phức tạp

Một CNN lớn nhìn ảnh thế nào?

Đây là phần **quan trọng nhất** của Giai đoạn 2. Ta dùng VGG-16 (đã pretrained trên ImageNet – 1.2 triệu ảnh, 1000 lớp) và đưa một bức ảnh con mèo vào. Ta sẽ **xem feature map** ở các lớp khác nhau:

- **Lớp đầu (block1):** feature map trông *rất giống ảnh gốc*, chỉ là các phiên bản phát hiện **cạnh, texture đơn giản**, các **vùng màu**.
- **Lớp giữa (block3):** feature map *trừu tượng hơn*, phát hiện các **patterns** và **texture phức tạp** (kẻ sọc, vảy lưới, đường cong).
- **Lớp sâu (block5):** feature map *rất ít liên hệ trực quan với ảnh gốc*; mỗi kênh đại diện cho một **khái niệm trừu tượng** (mắt, mũi, lông, miệng ...).

Cell 11: Tải VGG-16 đã pretrained và xem feature map

```
from torchvision import models
from torchvision.transforms import Compose, Resize, ToTensor, Normalize

# 1) Tải VGG-16 với weight ImageNet
vgg =
↳ models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1).to(device).eval()
print(vgg.features)           # Xem cấu trúc

# 2) Chuẩn bị ảnh con mèo (RGB, 224x224, normalize theo ImageNet)
transform_imnet = Compose([
    Resize((224, 224)),
    ToTensor(),
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
url = "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"
img_rgb =
↳ Image.open(io.BytesIO(urllib.request.urlopen(url).read())).convert('RGB')
x = transform_imnet(img_rgb).unsqueeze(0).to(device)
```

```
print("Input shape:", x.shape)
```

Cell 11 (tiếp): Trích feature map ở các lớp khác nhau

```
# Forward hook -- "móc" vào các tầng để lấy feature map
features = {}
def make_hook(name):
    def hook(module, inp, out):
        features[name] = out.detach()
    return hook

# VGG-16 có cấu trúc: features = Sequential(64 layers)
# block1_conv1 = features[0], block3_conv1 = features[10], block5_conv1 =
#   ↪ features[24]
vgg.features[0].register_forward_hook(make_hook("block1_conv1")) # Nông
vgg.features[10].register_forward_hook(make_hook("block3_conv1")) # Vừa
vgg.features[24].register_forward_hook(make_hook("block5_conv1")) # Sâu

with torch.no_grad():
    _ = vgg(x)

for k, v in features.items():
    print(f"{k}: feature map shape = {tuple(v.shape)}")
```

Cell 11 (tiếp): Vẽ 8 channel của mỗi lớp

```
def show_feature_maps(fmap, title, n_show=8):
    fmap = fmap[0].cpu() # bỏ batch dim
    fig, axs = plt.subplots(1, n_show, figsize=(16, 2.4))
    for i, ax in enumerate(axs):
        # Chọn 8 channel có activation lớn nhất (cho dễ nhìn)
        channel_means = fmap.mean(dim=(1, 2))
        top_idx = channel_means.argsort(descending=True)[:n_show]
        ax.imshow(fmap[top_idx[i]], cmap='viridis')
        ax.set_title(f"ch {top_idx[i].item()}", fontsize=9)
        ax.axis('off')
    plt.suptitle(title, fontsize=12)
    plt.tight_layout(); plt.show()

show_feature_maps(features["block1_conv1"],
                  "block1_conv1 (LỚP NÔNG) -- cạnh và vùng màu")
show_feature_maps(features["block3_conv1"],
                  "block3_conv1 (LỚP GIỮA) -- texture và pattern")
show_feature_maps(features["block5_conv1"],
```

```
"block5_conv1 (LỚP SÂU) -- bộ phận, khái niệm trừu  
→ tượng")
```

Sinh viên sẽ thấy gì

- **block1_conv1**: nhiều channel trông *rất giống ảnh con mèo*, chỉ là các phiên bản nhấn cạnh. Một channel sáng ở *viền lông*, một channel sáng ở *nền*, một channel sáng ở *cạnh ngang* ...
- **block3_conv1**: ảnh *trừu tượng hơn*. Vài channel hiện rõ *kẻ sọc, patterns lặp lại* của lông.
- **block5_conv1**: feature map *nhỏ hơn nhiều* (14×14) và *khó “đọc” trực tiếp*. Nhưng nếu nhìn kỹ, có channel sáng ở *vùng mắt mèo*, có channel sáng ở *vùng mắt*.

Đó chính là feature hierarchy: mạng tự xây dựng *thang trừu tượng* từ pixel thô lên khái niệm cấp cao.

Câu hỏi 11.1 – Feature Hierarchy

- (a) In ra shape của feature map ở 3 lớp. Em thấy: **càng đi sâu, kích thước không gian (H, W) càng _____, nhưng số kênh (C) lại càng _____**. (*Giảm / tăng.*) Tại sao? Đây là một quy tắc thiết kế phổ biến của CNN.
- (b) Ở **block1_conv1** có 64 channel. Mỗi channel có thể coi là một “*detector*” cho một loại pattern. Em **đếm thử**: trong 8 channel hiển thị, có bao nhiêu cái “phát hiện cạnh ngang”, bao nhiêu “phát hiện cạnh dọc”, bao nhiêu “phát hiện vùng màu sáng” ...
- (c) Thử thay ảnh đầu vào (tải một ảnh khác – ảnh chó, ảnh con người, ảnh ô tô). Xem feature map có thay đổi không? Ở lớp **nông** (block1) thay đổi nhiều hay ít? Ở lớp **sâu** (block5) thì sao?

2.4. Bonus: CNN Explainer – “Sờ tận tay” CNN

CNN Explainer (Polo Club, Georgia Tech)

Mở trình duyệt và truy cập:

<https://poloclub.github.io/cnn-explainer/>

Đây là một **trang web tương tác** cho phép sinh viên:

- Click vào từng pixel của ảnh đầu vào và xem nó ảnh hưởng tới feature map sau như thế nào.
- Xem trực tiếp phép tính tích chập 3×3 – từng số nhân với từng trọng số kernel, rồi cộng lại.
- Tải ảnh của riêng mình và xem CNN xử lý.
- Khám phá ReLU, Pooling, Softmax bằng giao diện trực quan.

Yêu cầu sinh viên **dành 10–15 phút** chơi với công cụ này. *Đây là cách hiệu quả nhất để hiểu trực giác CNN – không có công thức nào tốt hơn.*

Câu hỏi 12.1 – Sau khi chơi với CNN Explainer

- Mô tả **bằng lời** (không dùng công thức) cách một phép tích chập diễn ra: kernel $\begin{bmatrix} ?, ?, ? \\ ?, ?, ? \\ ?, ?, ? \end{bmatrix}$ đặt lên một vùng 3×3 của ảnh, rồi ... ?
- Khi đi từ Conv1 $\rightarrow$ Conv2 $\rightarrow$... feature map *thu nhỏ dần*. Vì sao? (*Hint: liên quan tới pooling và stride.*)
- Lớp **softmax** cuối cùng làm gì? (*Cho ra xác suất 10 lớp, tổng = 1.*)
- Có khái niệm nào trong CNN Explainer mà em **chưa hiểu rõ**? Ghi lại để hỏi ở buổi lý thuyết.

2.5. Tổng kết Giai đoạn 2 – Những gì sinh viên đã “thấy”

Bảng tổng hợp Trực giác CNN

Sau Giai đoạn 2, sinh viên **không cần biết công thức** mà vẫn *cảm nhận được*:

Khái niệm	Đã thấy gì?	Kết nối với ứng dụng
Kernel / Filter	Ma trận 3×3 áp lên ảnh ra ảnh mới (Sobel, blur, sharpen)	Mỗi kernel <i>phát hiện một loại pattern</i> . CNN có nhiều kernel cùng lúc.
Convolution	Trượt kernel qua ảnh, nhân-cộng	Là <i>trái tim</i> của CNN.
Feature map	Ảnh mới sau khi áp kernel	Là <i>cách CNN nhìn</i> ảnh.
Pooling	Ảnh thu nhỏ nhưng vẫn nhận ra	Giảm tính toán + <i>tăng vùng nhìn</i> .
Lớp đầu CNN	Phát hiện <i>cạnh, màu</i>	Giống Sobel + blur tự học được.
Lớp giữa CNN	Phát hiện <i>texture, pattern</i>	Lông, vảy, kẻ sọc ...
Lớp sâu CNN	Phát hiện <i>bộ phận, vật thể</i>	Mắt, mũi, mặt – mức trừu tượng cao.
Feature hierarchy	Cạnh → texture → bộ phận → vật thể	CNN tự xây thang trừu tượng.

Câu hỏi để mang đến buổi lý thuyết:

- Vì sao kernel có tham số được học? (Câu trả lời: backpropagation – buổi sau.)
- Công thức tính kích thước đầu ra của Conv là gì? (Buổi sau.)
- Padding và stride ảnh hưởng ra sao? (Buổi sau.)
- Số tham số của một CNN tính như thế nào? (Buổi sau.)

3. Bài tập về nhà – Mở rộng Giai đoạn 1 & 2

Hướng dẫn

Sinh viên **chỉ chọn 1 (một) bài tập** dưới đây. Yêu cầu nộp:

1. Notebook .ipynb có code và kết quả.
2. Báo cáo ngắn (1–2 trang PDF) trả lời các câu hỏi của bài.

Tinh thần: bài tập này tiếp tục theo phong cách *thấy trước – giải thích sau*. Sinh viên **không cần** đi sâu vào lý thuyết; chỉ cần *quan sát, ghi chép, đặt câu hỏi*.

3.1. Bài 1: Lặp lại WOW phase trên CIFAR-10

Thông tin

Dataset: CIFAR-10 – 60.000 ảnh 32×32 RGB, 10 lớp (máy bay, xe hơi, chim, mèo, hươu, chó, ếch, ngựa, tàu, xe tải). Có sẵn trong `torchvision.datasets.CIFAR10`.

Yêu cầu:

1. Lặp lại **Bước 2–6** (Bước 2: load data, Bước 3: định nghĩa CNN, Bước 4: train, Bước 5: vẽ learning curve, Bước 6: visualize prediction) cho CIFAR-10.
2. Sửa TinyCNN để nhận *3 kênh đầu vào* (RGB) thay vì 1 kênh: đổi `nn.Conv2d(1, 16, ...)` thành `nn.Conv2d(3, 16, ...)`.
3. Train 5 epoch. Báo cáo accuracy. (*Dự kiến: $\sim 60\text{--}70\%$ sau 5 epoch – thấp hơn MNIST nhiều!*)
4. Tìm 16 ảnh đoán sai. **Em có thể nhận ra chúng không?** Thường ảnh CIFAR mờ và khó hơn cả con người – ghi nhận cảm giác này.
5. Thêm augmentation (`RandomHorizontalFlip`, `RandomCrop(32, padding=4)`). Train lại 5 epoch. Accuracy có cải thiện không?

3.2. Bài 2: Tự thiết kế kernel và tạo “filter Instagram”

Yêu cầu:

1. Chọn 1 ảnh tự chọn (ảnh selfie, phong cảnh, vật nuôi, ...). Resize về 256×256 grayscale.
2. Áp dụng **6 kernel cổ điển** ở Cell 9 và lưu lại ảnh kết quả.
3. **Tự thiết kế** 3 kernel mới của riêng mình:

- Một kernel **phát hiện đường chéo**.
- Một kernel **tăng tương phản** (kết hợp sharpen + identity).
- Một kernel **thiên-vị-một-hướng** (vd: chỉ làm sáng phần bên phải).

Giải thích bằng lời *vì sao* kernel của em hoạt động như vậy.

4. **Bonus:** kết hợp 2 kernel liên tiếp (áp kernel A xong áp kernel B). Hiệu ứng có khác với một kernel duy nhất không?

3.3. Bài 3: So sánh feature map của 3 kiến trúc kinh điển

Yêu cầu:

1. Tải 3 mô hình pretrained từ torchvision.models:
 - alexnet(weights='IMAGENET1K_V1')
 - vgg16(weights='IMAGENET1K_V1')
 - resnet18(weights='IMAGENET1K_V1')
2. Chọn **cùng 1 ảnh** (ví dụ: ảnh con mèo) và lấy feature map ở **lớp tích chập đầu tiên** của mỗi mô hình.
3. Vẽ 8 channel của mỗi mô hình. So sánh:
 - **AlexNet:** 64 hoặc 96 channel, kernel 11×11 (rất to!).
 - **VGG-16:** 64 channel, kernel 3×3 .
 - **ResNet-18:** 64 channel, kernel 7×7 , stride=2.

Cảm giác channel đầu tiên trông *khác nhau* hay *giống nhau*? Mô hình nào có channel trông *giống Sobel/blur cổ điển nhất*?

4. **Phản tư:** dù ba kiến trúc rất khác, lớp đầu vẫn học những thứ **tương tự** (cạnh, màu, blob). Đây là một *"luật chung"* của CNN – đáng để ghi nhớ.

3.4. Bài 4: Khám phá CNN Explainer – viết blog ngắn

Yêu cầu:

1. Dành ít nhất 30 phút *chơi nghiêm túc* với <https://poloclub.github.io/cnn-explainer/>.
2. Viết **một bài blog ngắn** (1–2 trang) mô tả cảm nhận của em, dành cho **một bạn học sinh cấp 3 chưa biết AI**. Bài viết không được dùng công thức, chỉ dùng:
 - Hình ảnh chụp từ CNN Explainer.

- Phép so sánh đời thường (kernel = “kính lúp”, pooling = “thu nhỏ”, ...).
 - Câu chuyện: “Khi tôi tải ảnh chú mèo của tôi vào, mạng đã ...”
3. **Đây là bài tập rèn diễn đạt:** nếu em giải thích được CNN cho em mình hiểu, em đã thực sự hiểu CNN.

3.5. Bài 5 (nâng cao): Vẽ filter của lớp Conv1 sau khi train trên MNIST

Yêu cầu:

1. Train TinyCNN trên MNIST 5 epoch (đạt $\sim 98\%$).
2. Vẽ **16 filter** của `model.conv1` (mỗi filter là một ma trận $1 \times 3 \times 3$):

```
weights = model.conv1.weight.data.cpu()    # shape [16, 1, 3, 3]
fig, axs = plt.subplots(2, 8, figsize=(12, 3.5))
for i, ax in enumerate(axs.flat):
    w = weights[i, 0]
    ax.imshow(w, cmap='RdBu_r', vmin=-w.abs().max(), vmax=w.abs().max())
    ax.set_title(f"filter {i}", fontsize=8)
    ax.axis('off')
plt.suptitle("16 filter $3 \times 3$ mà CNN đã học được trên MNIST")
plt.tight_layout(); plt.show()
```

3. Cố gắng đặt tên cho từng filter dựa trên hình dáng:
 - Filter này nhìn *như Sobel-x* chẳng?
 - Filter kia có dạng *"cộng dọc"* ?
 - Filter này có dạng *góc trên-trái*?
4. **Phản tư:** mạng **không** được nói trước rằng cần học cạnh ngang, cạnh dọc, đường chéo ... Nó **tự khám phá** ra. Đây là điều kỳ diệu nhất của CNN.

3.6. Tiêu chí chấm điểm

Tiêu chí	Điểm
Code chạy được, có comment tiếng Việt, kết quả tái lập (set seed)	2.0
Hoàn thành đầy đủ các bước của bài đã chọn	2.5
Visualize đẹp, rõ ràng (biểu đồ, ảnh, feature map)	2.0
Quan sát + đặt câu hỏi: ghi nhận điều bất ngờ, đặt câu hỏi về điều chưa hiểu	2.0
Trình bày báo cáo dễ đọc, có cảm xúc cá nhân (<i>không phải copy lại lý thuyết</i>)	1.5
Tổng	10.0

Một lưu ý quan trọng về phong cách chấm

Khác với các lab khác, lab này **không yêu cầu sinh viên hiểu lý thuyết sâu**. Lab tốt là lab có nhiều **quan sát của riêng sinh viên** và nhiều **câu hỏi tự đặt ra**. Một câu *"em chưa hiểu vì sao ..."* viết kèm hình minh họa **quý hơn 10 dòng** paste lý thuyết. Lý thuyết sẽ học ở các buổi sau.

Tài liệu tham khảo & nguồn cảm hứng cho bài thực hành này

Repository GitHub đã được tham khảo:

- Utku Ozbulak – *PyTorch CNN Visualizations*: <https://github.com/utkuozbulak/pytorch-cnn-visualizations> – bộ sưu tập đầy đủ các kỹ thuật trực quan hóa CNN.
- Justin Bellucci – *CNN Visualizations PyTorch*: <https://github.com/justinbellucci/cnn-visualizations-pytorch> – code cài đặt VGG-16 visualization.
- Kush Madlani – *Visualising CNN feature maps*: <https://kushmadlani.github.io/visualising-cnn-layers/> – ý tưởng dùng forward hook để xem feature map.
- Francesco Saverio Zuppichini – *A journey into CNN visualization*: <https://github.com/FrancescoSaverioZuppichini/A-journey-into-Convolutional-Neural-Network-visualization->.
- PyTorch Tutorials – *60 Minute Blitz*: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html – nguồn cho phần WOW phase.

Công cụ tương tác:

- **CNN Explainer** (Polo Club, Georgia Tech): <https://poloclub.github.io/cnn-explainer/> – **rất khuyến nghị**, dùng trực tiếp trong buổi học.
- **TensorFlow Playground** (cho mạng nơ-ron tổng quát, không phải CNN): <https://playground.tensorflow.org/>.

Tài liệu lý thuyết (cho các buổi sau):

- Stanford CS231n: <https://cs231n.github.io/> – giáo trình CNN tốt nhất hiện có.
- Andrew Ng – *Edge Detection Examples* (deeplearning.ai): có trên YouTube và Coursera.
- Zeiler & Fergus, "*Visualizing and Understanding Convolutional Networks*", ECCV 2014 – bài báo gốc về feature hierarchy.

— Cảm nhận trước, lý thuyết sau. Chúc các em một buổi học vui! —